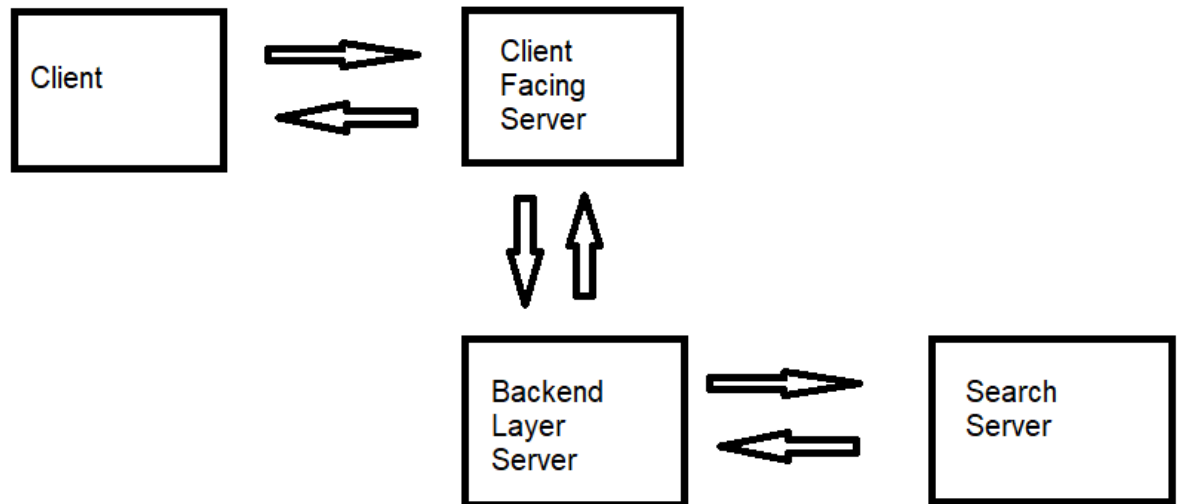


Software Engineering Final

Name: _____

- Exam is open note, but closed computer - printed notes are fine, but no laptop/phone/smart watch
- Please write legibly!
- Partial answers get partial credit - explain your reasoning to get credit for all the parts of the answer you know
- Write valid Java code for questions asking for code; you **do not** need to include any import statements (assume the IDE will handle that for you)

1. You've been asked to do a code review for the initial code to implement the following System Design Diagram:



What issue with how the code relates to the diagram could you flag as part of the review?

```
public interface ClientFacingServer {}
public interface BackendLayerServer {}
public interface SearchServer {}

public class ClientFacingServerImpl {
    private SearchServer searchServer;
    private BackendLayerServer backendLayerServer;

    public ClientFacingServerImpl(SearchServer searchServer,
        BackendLayerServer backendLayerServer) {
        this.searchServer = searchServer;
        this.backendLayerServer = backendLayerServer;
    }
}
```

```
public class BackendLayerServerImpl {
    private SearchServer searchServer;
    public BackendLayerServerImpl(SearchServer searchServer) {
        this.searchServer = searchServer;
    }
}

public class SearchServerImpl {
}
```

2. For this question, refer to the following interfaces and classes:

```
public interface LectureHall {

    public void addTextToBoard(String text);
    public void clearSpaceOnBoard(int squareFeetToClear);

    public String getCurrentText();
}
```

```

public interface Whiteboard {
    public void addText(String text);
    public void eraseLineOfText();
}

public class LectureHallImpl implements LectureHall {

    private Whiteboard leftBoard;
    private Whiteboard rightBoard;

    public LectureHallImpl() {
        leftBoard = new WhiteboardImpl();
        rightBoard = new WhiteboardImpl();
    }

    /* Method implementation omitted */

}

public class WhiteboardImpl implements Whiteboard {
    /* Method implementation omitted */
}

```

Imagine you're writing a unit test for the LectureHallImpl class.

- (a) What design pattern could you use to change LectureHallImpl to make the unit test easier to write?

(b) What would the code for the updated LectureHallImpl be? You only need to re-write the parts of the class that would change.

3. Consider the following code. Draw a picture of what the stack and heap would look like just after executing the last line of createBasicShelf, but before returning from the method (you can assume that initialBook is not null):

```
public class Bookshelf {  
    private Shelf[] shelves;  
    public Bookshelf(int numShelves) {  
        shelves = new Shelf[numShelves];  
    }  
    public void addBookToShelf(int shelfNum, Book book) {  
        shelves[shelfNum].addBook(book);  
    }  
}
```

```
public class Shelf {
    private static final int SHELF_WIDTH = 5;
    private Book[] books = new Book[SHELF_WIDTH];
    private int bookIndex = 0;
    public void addBook(Book book) {
        books[bookIndex] = book;
        bookIndex++;
    }
}

public class Book {
    private String title;
    private String contents;
}

public void createBasicShelf(Book initialBook) {
    Bookshelf bookshelf = new Bookshelf(3);
    bookshelf.addBookToShelf(1, initialBook);
}
```

4. Consider the following code to simulate running a lottery:

```
public interface Player {
    void pickNumberToBet();
    void collectWinnings();
    boolean hasWon(int winningLottoNumber);
}

public void lotterySimulator(List<Player> lottoPlayers) {

    // Step 1: Each player picks the number they want to bet on
    for (Player p : lottoPlayers) {
        p.pickNumberToBet();
    }

    // Step 2: The winning number gets picked
    int winningLottoNumber = pickWinningNumber();

    // Step 3: Each player checks to see if they've won
    for (Player p : lottoPlayers) {
        if (p.hasWon(winningLottoNumber)) {
            p.collectWinnings();
        }
    }
}

private int pickWinningNumber() {
    // method implementation omitted
}
```

Modify the code so that step 1 and step 3 run in parallel for each player. Step 2 must still come after step 1 and before step 3.

// More space for Q4

5. Imagine you're working on a system that has a proto message that used to look like:

```
message PersonMessage {  
    optional string firstName = 1;  
    optional string lastName = 2;  
}
```

and now looks like:

```
message PersonMessage {  
    // deprecated fields, prefer combining names into fullName  
    optional string firstName = 1;  
    optional string lastName = 2;  
    optional string fullName = 3;  
}
```

On the server, there's a class Person that looks like:

```
public class Person {  
    private String fullName;  
  
    public LookupRequest(String fullName) {  
        this.fullName = fullName;  
    }  
  
    public String getFullName() {  
        return fullName;  
    }  
}
```

Write a method that will take in a parameter of type PersonMessage (auto-generated from the proto), and return an equivalent Person, making sure to correctly handle both old and new style proto messages.

6. Consider the following class that handles caching imagery from the internet locally. What's a problem that a program using this cache might experience, and what code could you add to the class to fix that problem?

```
public class ImageCache {
    private static Map<String, byte[]> cachedImages =
        new HashMap<>();

    public byte[] fetchImage(String url) {
        if (cachedImages.containsKey(url)) {
            return cachedImages.get(url);
        }

        byte[] image = downloadImage(url);

        cachedImages.put(url, image);
        return image;
    }

    private byte[] downloadImage(String url) {
        // implementation omitted
    }
}
```

7. Imagine you're building a system that will take in a stream of music artists and determine the two most-played artists from that stream. Write a method to do an initial transformation of the incoming stream (ie, a `map` method), and a method that will combine either batches or pairs of results (ie, a `reduce` method) that could be combined with a MapReduce framework to calculate the two most-played artists.

```
public interface Artist {  
    int getNumberTimesPlayed();  
    String getName();  
}
```